

Architectural Patterns for Integrating LLMs into User-Facing Applications

Lessons from a language-learning platform

MIRCEA LUNGU, IT University of Copenhagen, Denmark

CESARE PAUTASSO, University of Lugano, Switzerland

Large Language Models are increasingly being integrated as components into existing user-facing applications, alongside the more visible wave of LLM-native products such as chatbots and agents. The engineering concerns of the two cases differ: when an LLM is a component behind an existing feature rather than the product itself, designers must reconcile its per-token cost, multi-second latency, non-determinism, rapidly shifting provider landscape, and general-purpose capability with the expectations of users who already rely on the system. We present a catalogue of recurring architectural patterns that address these concerns, grounded in over a year of LLM integration work on Zeeguu, an open-source language-learning platform with several hundred monthly active users. The catalogue spans five concerns — cost optimization, latency and availability, lifecycle management, data management, and quality assurance — and is described using the standard pattern format. The patterns are presented as a starting point for community refinement and extension, not as a closed taxonomy.

Additional Key Words and Phrases: large language models, software architecture, design patterns, LLM integration, cost, latency, quality assurance

1 Introduction

While there is growing literature on building LLM-native products (chatbots, agents, RAG systems) and on using LLMs for code generation, there is surprisingly little guidance on the **software engineering challenges of integrating LLMs as components into existing interactive applications** where real users expect fast, reliable, and trustworthy responses.

We expect this integration to become the common case: over time, more and more existing user-facing applications will adopt LLMs not as standalone chatbots but as components working behind the scenes to improve the user experience.

LLMs have a unique combination of properties that create novel architectural forces:

- they are expensive (per-token pricing),
- slow (high latency),
- non-deterministic (same input can yield different outputs, although this can be to a certain degree controlled with the temperature setting),
- rapidly evolving (new models released every few months and old ones regularly deprecated),
- imprecise (they make mistakes), and
- general-purpose (they can attempt almost any task described as text).

These properties demand specific engineering strategies.

Over the past year, we have been integrating LLMs into Zeeguu¹, an open-source platform for personalized language learning that helps users learn foreign languages by reading authentic online content. Through this work, we have

¹<https://zeeguu.unibe.ch/>

identified a set of recurring architectural patterns for LLM integration. We believe these patterns are general and applicable beyond our specific domain.

2 Case Study: Zeeguu

Zeeguu is an open-source language learning platform built around the idea that learners benefit most from engaging with authentic content in their target language. Rather than relying on artificial textbook exercises, Zeeguu helps users find real articles (news, blog posts, and other web content) tailored to both 1) their *level* and 2) their *interests*.

The platform recommends articles in the learner’s target language based on their proficiency and reading preferences, making it **easy to find material that is both engaging and appropriately challenging**. If a text is personally compelling but too difficult, Zeeguu simplifies it to the learner’s level using LLMs.

When users encounter unfamiliar words or phrases while reading, they can get translations on the fly powered by (contextual) Google Translate, so the reading experience remains fluid and uninterrupted. One alternative is provided by a state-of-the-art LLM, offering a more contextually nuanced option.

Every translation a user requests is logged by the system, which over time builds a **detailed model of the learner’s vocabulary knowledge**, tracking which words they know, which ones they struggle with, and how well they’ve retained previously encountered vocabulary.

Based on this evolving learner model, Zeeguu **generates interactive vocabulary exercises and audio lessons** that focus on the words that matter most for each individual learner, rather than following a generic curriculum. The exercises use the context in which the word was originally encountered, based on the assumption that if the original text was compelling to the learner, examples drawn from it will be too.

In essence, Zeeguu unifies reading, translation, learner modeling, and practice into a coherent pipeline, with the learner’s own reading interests as the primary driver.

2.1 The Pieces

A handful of Zeeguu-specific concepts recur across the patterns. They are collected here once, so a pattern can point back to a single definition rather than re-explaining them.

2.1.1 CEFR Levels. The Common European Framework of Reference grades language proficiency on an ordered six-level scale, from A1 (beginner) to C2 (mastery). Zeeguu uses it in two directions: to estimate how hard an article is, and to simplify an article down to every level easier than the original.

2.1.2 Article Simplification. When an article is compelling but too hard, Zeeguu rewrites it with an LLM to easier CEFR levels, producing one simplified version per level below the original. It runs both on demand, when a reader opens an article that has not been simplified yet, and ahead of time, over the crawled feed.

2.1.3 Crawling. Zeeguu builds its article recommendations by crawling news sites and blogs multiple times per day; this steady feed of freshly crawled articles is where most ahead-of-time LLM work happens (CEFR assessment, simplification), off any reader’s critical path. On demand, readers push their own content in: a browser extension sends any article to Zeeguu for study, and on mobile the system share sheet sends any web page to be made interactive.

2.1.4 Multi-Word Expressions. A multi-word expression (MWE) is a group of words whose meaning is not the sum of its parts, for example *break the ice*. Zeeguu detects them so a learner can translate the phrase as a unit rather than word by word. A cheap dependency-parse gate (Stanza) fires first, and an LLM confirms only the flagged sentences.